

试 题

题号	一	二	三	四	五	六	七	八	总分
分数									

1. 考试形式：闭卷 ☒ 开卷 ☐

2. 考试日期： 年 月 日 (答题内容请写在装订线外)

一、关于分布式系统的概念和模型，回答下列问题。(共 20 分)

1. 设计分布式系统时通常采用消息传递模型，简述消息传递模型的含义。(4 分)
2. 消息传递模型与设计多线程并行计算系统时采用的模型有何不同？(2 分)
3. 要设计能够真正用于实际生产环境的多计算节点分布式协作系统，必须要考虑容忍哪几类错误？(6 分)
4. 简述失效停止节点错误模型和拜占庭节点错误模型的区别？(4 分)
5. 要利用分布式计算系统对一个大规模数据集进行排序，你所知道的排序算法中哪一种更适合？为什么？(4 分)

1. 节点之间没有公共状态，必须通过相互发送消息来进行协作

2. 多线程并行计算采用共享存储模型，对于消息传递模型来说，通信复杂度是影响效率的重要因素。

3. 对应 CAP (?)

网络分区错误 (P) 节点故障与服务不可用 (A) 数据不一致问题 (C)

4. 失效停止节点错误模型：节点故障就停止工作，故障期间不发送任何消息。

拜占庭节点错误模型：节点一旦发生故障就可能做出任意动作，可能发送错误消息甚至恶意发送消息影响分布式系统，更难被检测

5. 归并排序，适合划分成不同子任务进行处理，子任务之间不需要交换信息，执行效率高，最后再归并总的结果。

并行



二、关于负载均衡技术，回答下列问题。（共 13 分）

1. 要实现一个支持高并发访问的 Web 网站系统，如何利用负载均衡技术提高该网站的水平可扩展性？如何提高网站的可用性？（4 分）

2. 列举 5 种常用的负载均衡策略。（5 分）

3. 在用服务器集群实现 Web 服务系统时，如果完成实际服务运算的后端服务器都实现了缓存机制，那么采用哪种负载均衡策略更适合？为什么？（4 分）

1. ①利用负载均衡器将用户请求均匀发到多个后端服务器上，提升网站的并发处理能力，并支持水平扩展（增加服务器数量）
②提高可用性：负载均衡器可监控后端服务器的健康状态，当某个节点发生故障时自动将流量切换到其他健康结点。

2. 随机、轮询、固定权重值、IP 哈希、最小 TCP 连接数
最小响应时间、基于各服务器实际负载的动态负载均衡算法

3. IP 哈希，可以让同一 IP 的用户请求持续访问同一服务器，该 IP 先前的数据已经在服务器缓存，可避免重复处理，减少服务器开销，提升处理效率



三、关于分布式节点之间的通信技术，回答下列问题。（共 12 分）

1. 简述^①远程过程调用 RPC 的实现原理和^②RPC 中间件的主要作用。（4 分）
2. 互联网电商平台在应对秒杀、双十一突发访问峰值等场景时如何利用分布式消息中间件提高系统的高并发服务能力？（4 分）
3. 除了 RPC 和消息队列通信方式，分布式系统中不同节点之间还可以采用哪些通信方式？（列举 2 种）（4 分）

1. ① RPC 中间件在调用者进程中植入 stub/proxy 模块，stub 模式作为远程过程的本地代理，并暴露与远程过程相同的接口；

被调用者进程植入 skeleton 模式，作为调用者在远程主机中的代理，代替客户端调用本地方法，并把结果返回给客户端；

stub 模块与 skeleton 模块利用 socket 进行通信

② 定义并实现远程过程调用协议；

实现了过程参数和过程运算结果的序列化，反序列化；

实现了通信过程中的错误处理；

过程服务进程的集中注册与发现；

远程对象的统一标识和生命周期管理；

在服务端支持并发访问

2. ① 将大量瞬时用户请求异步写入消息队列

② 异步处理用户消息，先回应用户再进行后续处理

③ 系统解耦，提升容错能力，各模块独立工作

④ 水平扩展

3. 共享存储，socket，webservice



四、关于分布式共识和分布式一致性模型，回答下列问题。（共 9 分）

1. 在一个包含 n 个节点的分布式系统中应用 Paxos 或 Raft 分布式共识协议实现不同节点间的数据同步，最多可以容忍多少个节点的失效？（3 分）
2. 某个支持多副本模式的分布式存储系统采用了如下设计：客户端的写入操作都发送给主节点，主节点写入本地存储的同时将该请求广播给所有副本从节点。主节点等到所有副本节点返回同步成功的消息之后再给该客户端返回写入成功消息。如果某个副本节点超过规定时间没有响应，主节点就反复重试。客户端的读取请求可以发送给主节点或任意的副本从节点。请问该系统设计是否满足强一致性模型？是否满足最终一致性模型？为什么？（6 分）

1. 可以容忍 $\lfloor \frac{n-1}{2} \rfloor$ 个节点失效，因为系统必须有超过一半以上的节点同意才能提交操作

2. 不满足强一致性，比如主节点向所有从节点广播写请求，从节点 A 收到广播已经修改完成，从节点 B 有延迟还没收到，这时：客户端向从节点 A 读到更新后的数据，再向从节点 B 读取，读到旧数据，这一事件发生在读新数据之后，因此不满足强一致性

满足最终一致性，因此有节点未响应，主节点会反复重试，最终可达到一致状态。



五、关于 HDFS 分布式存储系统，回答下列问题。（共 10 分）

1. 简述客户端读取一个 HDFS 文件时与 NameNode 节点和 DataNode 节点的交互过程。（4 分）
2. 为了提高数据存储的可靠性，HDFS 在设计上采取了哪些技术措施？（至少给出 2 种）（4 分）
3. 为什么 HDFS 不适合存储大量的小文件？（2 分）

1.
 - ① 客户端向 NameNode 发送读请求，包含文件名、偏移量、长度等。
 - ② NameNode 根据信息查找内存中“文件名-数据块对应表”和“数据块-物理节点对应表”，并将包含目标数据块的数据节点 IP 列表发送给客户端。
 - ③ 客户端从 Datanode IP 列表选择“最近的 Datanode”，并与该节点建立 Socket 连接读取数据。
2.
 - ① 采用数据多副本方案，一个数据块的多个副本存不同 Datanode。
 - ② 每个数据块带校验和，客户端和 Datanode 读写时校验。
3.
 - ① NameNode 建表，运行开销大。
 - ② 每个文件至少占一个 block，一个 block 存储空间较大，易造成浪费。
存



六、针对以下分布式数据库设计场景回答问题。(共 12 分)

场景描述：假设某分布式数据库系统根据新插入数据记录的主键(key)选择不同的服务器节点存储该记录，共有 N 个存储服务器，标识号分别为 $ID_1, ID_2, \dots, ID_N$ 。另外假定存在理想的无碰撞哈希函数 H ，能够将任意的字符串转化为 5bit 的正整数， H 的输出结果在区间 $[0, 31]$ 上均匀分布。向该数据库共插入了 10 条记录，这 10 条记录的主键的哈希值分别为 1, 3, 5, 9, 14, 15, 19, 21, 26, 31。

1. 假定用以下策略选择具体的存储服务器：计算 $i = H(\text{key}) \bmod N$ ，其中 $\bmod$ 表示取余运算，然后将当前记录存储在服务器 ID_{i+1} 中。初始时 $N=3$ ，当增加一个服务器 ID_4 时会有多少条数据记录需要在不同服务器节点间移动？给出计算过程。(4 分)。
2. 设 $N=3$ ，并且 $H(ID_1)=4, H(ID_2)=23, H(ID_3)=13$ 。如果采用基于一致性哈希算法的服务器选择策略，那么这三台服务器分别存储了几条记录？给出计算过程。(4 分)
3. 接上题，仍然采用基于一致性哈希算法的服务器选择策略，当增加一个服务器 ID_4 (已知 $H(ID_4)=30$) 时，有多少条数据记录需要在不同服务器节点间移动？(4 分)

1. 移动前

ID_1 3 9 15 21

ID_2 1 19 31

ID_3 5 14 26

移动后

ID_1

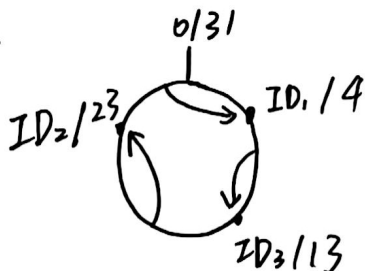
ID_2 1 5 9 21

ID_3 14 26

ID_4 3 15 19 31

需移动 7 条数据记录

2.



ID_1 1 3 26 31

4 条记录

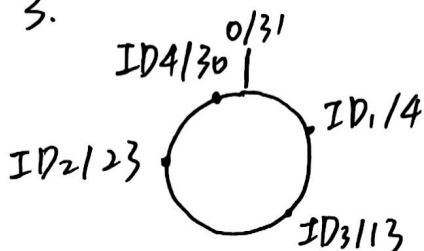
ID_3 5 9

2 条记录

ID_2 14 15 19 21

4 条记录

3.



移动后

ID_1 1 3 31

ID_3 5 9

ID_2 14 15 19 21

ID_4 26

需迁移 1 条记录



七、根据需求，描述 MapReduce 程序的实现方法。(12 分)

假定作为输入的 HDFS 数据文件中保存了某公司员工的工资信息，格式如下：

员工姓名 1, 部门 1, 年份 1, 月份 1, 工资金额 1,
 (注:
 为换行符)

员工姓名 2, 部门 1, 年份 1, 月份 1, 工资金额 2,

员工姓名 3, 部门 2, 年份 1, 月份 1, 工资金额 3,

员工姓名 4, 部门 3, 年份 1, 月份 1, 工资金额 3,

.....

员工姓名 1, 部门 1, 年份 1, 月份 2, 工资金额 4,

员工姓名 2, 部门 1, 年份 1, 月份 2, 工资金额 5,

.....

已知该公司 2021 年没有人员流动。现要求编写 MapReduce 程序，统计不同部门员

工 2021 年的年平均工资，即输出如下形式的统计数据：

部门 1, 员工年平均工资 1

部门 2, 员工年平均工资 2

.....

用伪代码描述该 MapReduce 程序的主要设计思想。

分两次进行 MapReduce 操作 语法很抽象我也不知道是啥语言

def mapper1(self, -, line):

for (员工姓名, 部门, 年份, 月份, 工资金额) in readline(): // 逐行读入数据

if 年份 == 2021:

yield 员工姓名, (部门, 工资金额) // 以员工姓名为 key,

(部门, 工资金额) 为 value
发送给 reducer 函数

def reducer1(self, key, values):

total = 0

for 工资金额 in values:

total += 工资金额

yield key, (部门, total) // 输出为 姓名, 部门, 2021 年总工资

第一次 Map 先计算
员工个人 2021 年总工资

第二次以部门为 key
计算年平均工资

def mapper2(self, -, line):

for (部门, total) in readline: // 逐行读入, 只关注部门和总工资

yield 部门, total

def reducer2(self, key, values) // values 是一个集合, 所有 key 相同
对应的 value 的集合

all = 0

for total in values:

all += total

// all 是部门总工资, total 是个人年总工资

avg = all / sum(values)

yield key, avg

↑ ↑
部门 员工年平均工资



八、根据需求，描述 Spark 程序的实现方法。（12 分）

假定作为输入的 HDFS 数据文件中保存了一个大规模的有向图，该文件每一行都存放了一个形如<ID1, ID2, Weight>的三元组，表示一条以节点 ID1 为始点、节点 ID2 为终点的有向边，Weight 是边的权值。现要求设计基于 Spark 并行计算模型的程序对该文件进行分析，获得所有度数（出度+入度）大于 1000 的节点的标识。用伪代码描述该 Spark 程序的主要思想。

```
rdd = textfile(input_file) // 读入文件

rdd_out = rdd.map(lambda ID1: (ID1, 1)) // 出度, 只考虑 ID1
                                key  value

rdd_in = rdd.map(lambda ID2: (ID2, 1)) // 入度, 只考虑 ID2

rdd_all = rdd_out.union(rdd_in) // 两份键值对合并, 计算总度数

rdd_count = rdd_all.reduceByKey(lambda a, b: a+b) // 计算总度数
            类似 reduce 函数, 对于 key 相同的键值对, a, b 对于两个键值对的 value
            将其相加再存入, 最终得到总度数
            value1 value2

rdd_result = rdd_count.filter(lambda value: value > 1000) // 筛选度数大于 1000 节点

result = rdd_result.collect() // 收集打印结果
```

